

Heizen Engineering Assignment: Kitchen Operations Admin Panel

Time: 48 hours · **Submission:** Git repository link(s) plus a live deployed link

1. The business

Fernleaf Kitchen (a fictional company) is a commercial kitchen that runs **corporate meal programs**. Companies sign up, their employees order individual boxed meals for specific delivery dates, and the kitchen cooks, packs and delivers them to the company's office. Employees never pay: every order is billed to their company.

All of this runs on one internal admin panel used by the kitchen's own staff. **You are building that admin panel.**

There is no customer-facing app in this assignment. Staff create orders on behalf of employees inside the admin panel, and that is enough to exercise every workflow.

Read the whole document before you start. The rules in section 4 matter more than the screens.

2. Ground rules

Stack (mandatory)

Layer	Use
Frontend	Next.js
Backend	NestJS
ORM	Prisma

- Beyond these three, you choose: repository structure (monorepo or separate repos), UI library, state and data fetching, validation, auth approach, styling and testing. Be ready to defend each choice.
- **Database:** we do not provide one. Use any database Prisma supports.
- The frontend must talk to the backend over HTTP. Do not put business logic in Next.js server actions that bypass the API.

Live deployment

We review your work by signing in to your live link. Deploy it anywhere you like (free tiers are fine) and keep it running for at least two weeks after you submit.

Create these accounts on the live app, with exactly these credentials. We will use them to sign in and test.

Role	Email	Password
Admin	admin@test.com	Test@1234
Kitchen	kitchen@test.com	Test@1234
Dispatch	dispatch@test.com	Test@1234
Driver	driver@test.com	Test@1234

Each account must have only that role's access.

Data

The live app must **already contain realistic data** when we open it. We judge the product by clicking through it, so empty screens hurt you. At minimum:

- several companies, each with employees
- a real-looking menu
- orders in every status, spread across past dates, **today** and the coming week
- deliveries assigned to `driver@test.com` for today

How you create the data is up to you. Keep in mind that "today" will be whichever day we review.

What we expect

- **The data model is yours.** Designing it is one of the main things we evaluate.
- **AI tools are allowed.** You must be able to explain every line you submit.
- **Scope is deliberately larger than the time you have.** Prioritising is part of the test. See section 6.

3. Who uses the panel

Role	What they do day to day
Admin	Everything. Sets up the catalogue, pricing, companies and employees, creates orders, and can override anything.
Kitchen	Sees what has to be cooked and marks work as started and done. Read-only on everything else they need.
Dispatch	Moves cooked orders out of the door, assigns drivers and tracks delivery.
Driver	Sees only their own deliveries for the day and marks them delivered, with a note and optionally a photo.

- Each staff member has exactly one role. Admins create staff accounts and assign roles.
- Permissions must be **enforced on the server**. Hiding a button is not access control.
- Adding a new role later should not mean hunting through the codebase for role-name checks.

4. Functional requirements

Requirements are tagged **[Must]**, **[Should]** or **[Could]**.

4.1 Catalogue **[Must]**

- **Dishes.** A dish has a name, description, image, an internal SKU number, a temperature (hot/cold), a cost price, allergens, dietary tags (vegan, Jain, gluten-free, ...), a kitchen station and an optional minimum order quantity. Dishes are deactivated, never hard-deleted, because historical orders reference them.
- **Options.** Reusable choices (paneer, tofu, jeera rice, raita, mint chutney) with their own cost, allergens and dietary tags.
- **Option groups.** A dish has zero or more groups of options, for example "Choose your protein: paneer, tofu or chickpeas". A group can be required or optional, has a display order, and lists which options it offers, in order.
- **Portions **[Should]**.** Some groups sell options in sizes (regular, large), each with an extra charge on top of the option's own price. A group either uses portions or it doesn't. If it does, every option in it must support the group's sizes.
- **Reference data.** Allergens, dietary tags, kitchen stations and portion sizes are admin-managed lists.

How options are ordered. An order line for a dish is split into **combinations**, each with its own quantity.

Example: 10 paneer rice bowls, of which 6 are with brown rice and 4 with jeera rice.

- The combination quantities must add up exactly to the dish quantity.
- Every combination must satisfy every required group.
- Price of a combination = (dish price + chosen option prices) × its quantity.
- The kitchen cooks each distinct combination as one unit (4.7).

An order line must keep a record of what was ordered **and at what price**. Editing the catalogue or prices later must never change a past order.

4.2 Menu **[Must]**

- Dishes are shown to employees through **categories** ("Bowls", "Breakfast", "Desserts"). Categories and their items are ordered and can be activated or deactivated.
- **Hiding.** A category or item can be hidden from specific companies. A category can also be *secret*, meaning it is not listed but can still be reached.
- Staff need a way to **preview the menu exactly as a given employee would see it**. The preview must apply the rules above, plus pricing (4.3).

4.3 Pricing **[Must]**

The same dish costs different amounts for different companies. Build a pricing system that meets these requirements:

1. There are several named **price tiers** (e.g. "Standard", "Enterprise", "Partner"). Each dish and each option can have a price on each tier.

2. One tier is the **default**.
3. A company can be put on a different tier.
4. An employee's price is worked out from their company's tier, or from the default tier if the company has none.
5. **A dish with no price on the employee's tier must not appear on their menu at all.** It should not show at \$0 or with a blank price.
6. A tier can **derive** its prices from another value, for example "cost × 2.4" or "Standard price + 15%", instead of having every price typed in. Staff can still override individual prices. Derived prices **round up to the next 5 cents** (\$2.11 becomes \$2.15).
7. Staff need a fast way to see and edit a whole tier, and to spot dishes that have no price on a tier.
8. Changing a price affects new orders only (see 4.1).

We intentionally don't tell you how to model or store any of this.

4.4 Companies [Must]

- A company has a name, one or more **email domains** (two companies cannot claim the same domain, and public domains such as gmail.com are not allowed), one or more delivery addresses, billing contact details and an owner, who is one of its employees.
- **Calendar.** Working days (default Mon–Fri) and company holidays. The company can't receive deliveries on non-working days or holidays. **The company calendar does not move the cut-off.** Only the kitchen calendar does (4.6).
- **Delivery defaults.** A default delivery time, how many minutes before delivery the order must leave the kitchen (default 60), a default packaging type, standing instructions for the driver, and a default driver.
- **Menu and price.** The company's price tier (4.3) and hidden categories or items (4.2).

4.5 Employees [Must]

- Every customer is an employee of exactly one company. Moving an employee to another company changes which rules apply to them.
- Each employee has permission flags set by staff: can choose their own delivery address, can change the delivery time, and can change packaging.
- Employees have allergies and dietary preferences.
- **[Should]** Bulk-import a company's employees from a CSV file. Report row-level errors without rejecting the whole file.

4.6 Orders [Must]

Cut-off. Orders for a delivery date lock at a configured time, a configured number of *kitchen working days* before delivery.

- Example: with a cut-off of 2 working days at 16:00, a Wednesday delivery locks at Monday 16:00.
- Kitchen holidays and non-working days are skipped when counting back.
- The cut-off time and day count are settings (4.10).

Creating. Staff create an order for an employee. The flow:

- choose a delivery date
- see that employee's menu (4.2, 4.3)
- add dishes with valid option choices (4.1)
- choose address, time and packaging where the employee is allowed to
- see the price breakdown per line and the order total
- place it

Every rule in this document must be **validated on the server**, not only in the form. Staff can also save an order as a **draft**.

Statuses. Draft → Placed → Confirmed → Delivered, plus Cancelled and Rejected. Before the cut-off, Draft and Placed orders can be edited and cancelled. After the cut-off they cannot, except by an admin.

Cut-off processing. When the cut-off for a delivery date passes:

- Every **draft** for that date is cancelled.
- Every **placed** order for that date is confirmed and becomes billable to its company (4.9).
- Running cut-off processing twice for the same date must be safe.
- There must be a way to trigger it manually for a past cut-off, so a reviewer can try it without waiting.

Order list and detail.

- A searchable, filterable, paginated list, filtered at least by delivery date range, status, company and whether the order has been invoiced.
- The detail page shows the lines, option choices, money breakdown, delivery details, and a timeline of the order's progress.

Admin overrides. Admins can change an order's delivery time, address or packaging after confirmation.

4.7 Kitchen board [Must]

- Shows what has to be cooked for a chosen delivery date, broken into **prep units**. Each distinct combination on an order line is one unit (4.1). A unit is routed to its dish's kitchen station, or to "Unassigned" if the dish has none.
- Kitchen staff filter by station and mark each unit **started** and **done**:
- Only confirmed orders can be worked on.
- A unit can't be started twice or finished twice.
- Finishing a unit that was never started is allowed and also records a start.
- The order's "kitchen started" time is its first unit's start. Its "kitchen ready" time is set only when every unit is done.
- Each order has a **planned kitchen-ready time** and a **planned dispatch-ready time**, worked back from the delivery time:
 - dispatch-ready = delivery time minus the company's delivery minutes
 - kitchen-ready = dispatch-ready minus 30 minutes
- The plan updates if the delivery time changes. The board must make late and at-risk work obvious.

- An admin can force-complete a whole order.

4.8 Dispatch board and driver view [Must]

- Orders move through **kitchen ready** → **dispatch ready** → **out for delivery** → **delivered**. Each step requires the previous one and can't be repeated. "Out for delivery" also requires an assigned driver.
- **Drop grouping**. Orders for the same company, same address and exact same delivery time make up one drop and are handled together.
- Dispatch assigns a driver per drop (defaulting to the company's default driver) and sees each drop's status at a glance.
- **Driver view**. A driver signs in and sees only their own drops for today, in time order. They mark each one delivered with an optional note and photo. Make it usable on a phone.
- Record whether each delivery was on time.

4.9 Company billing [Must]

- Every confirmed order is owed in full by the employee's company.
- Staff can see, per company, every confirmed order not yet invoiced, group them into an **invoice** (an internal record, with no accounting integration), and mark invoices paid.
- An order can be on at most one invoice.
- Orders can still change after confirmation (admin overrides, cancellation, a delivered order that turns out short). Decide what happens to an order that is already invoiced, and document it.

4.10 Settings [Must]

Kitchen working days, kitchen holidays, the cut-off time and day count, and any other platform-wide values your design needs. Staff should not have to edit code or the database by hand to change them.

4.11 Dashboards [Must]

Each role lands on a dashboard after signing in.

We will not tell you what goes on them. Think about what each person (an admin, a kitchen lead at 6 am, a dispatcher, a driver) needs to see to do their job or make a decision. Then build it.

In your README, list for each dashboard:

- what you show, and why that person needs it
- exactly how each figure is calculated, including which orders count, which date it is grouped by, and how cancelled orders and missing data are treated
- what you chose *not* to show

We value well-defined, honest figures over impressive-looking charts.

5. Out of scope

Do **not** build any of the following. Where a feature depends on one, simulate it as described.

Out of scope	What to do instead
Payments by employees (cards, charging, refunds)	Every order is billed to the company (4.9).
Multiple order types (family style, catering trays, ...)	There is one kind of order: individual boxed meals.
Individual customers without a company	Every customer belongs to a company.
Options included free with a dish	Every option is chosen.
Date-based menu scheduling (seasonal or festive menus)	Items are shown or hidden only by the active flag and company hiding.
Pausing an employee's ordering	Not required.
Exports (CSV, prep sheets, labels, reports)	Not required.
Accounting software integration	Invoices are internal records only.
Recipe or costing software integration	Costs are entered by hand in the catalogue.
Promotional and coupon codes	Not required.
Sales tax calculation	No tax. Totals are pre-tax.
Delivery fees and delivery zones	No fees. The order total is the sum of its lines.
Audit logs	Not required.
Customer-facing ordering app	Staff create orders in the admin panel.
Email and notifications	Not required. Logging to the console is fine where an email would be sent.
Marketing campaigns and banners	Not required.

6. Prioritisation

You will not finish everything. We would rather see **[Must]** items done properly than everything done badly.

In your README, include:

- what you built, what you skipped, and **why**
- what you would do next with more time
- any requirement you thought was ambiguous, and how you interpreted it

Interpreting ambiguous requirements sensibly and writing the interpretation down is part of the test.

7. Non-functional requirements

- **Correctness of money.** Don't let floating-point errors into prices or totals. Totals must reconcile: the order total equals the sum of its lines, and an invoice total equals the sum of its orders.
 - **Time zones.** The kitchen operates in one time zone. Cut-offs, delivery dates and "today" must be correct regardless of the server's or the browser's time zone. State which time zone you assumed.
 - **Concurrency.** Two staff members acting on the same order or kitchen unit at the same moment must not corrupt it. For example, two people might mark the same unit done.
 - **Validation.** Input is validated on the server, and errors are shown to the user in a form they can act on.
 - **Performance.** Lists are paginated on the server. The kitchen board for a busy day (assume 400 orders) must stay responsive.
 - **Code quality.** Clear module boundaries, types shared between frontend and backend where it makes sense, and consistent error handling. The repo must lint and type-check cleanly.
 - **Tests.** Test the business rules most likely to break: cut-off calculation, pricing resolution, combination counting and invoicing. We do not expect high coverage of UI code.
-

8. Deliverables

1. **A live link**, with the four test accounts from section 2 working.
 2. **Git repository link(s)**, public or shared with us, with a clean commit history. We read the history.
 3. **A README.md** with: - local setup instructions - an architecture overview and data model diagram - key decisions and trade-offs - the dashboard definitions (4.11) - the prioritisation notes (section 6)
-

9. How we evaluate

Area	What we look for
Domain modelling	Does the data model reflect the business? Are the rules in section 4 actually enforced? Will it survive the next requirement?
Correctness	Cut-offs, pricing and money are right, including edge cases.
Product thinking	Dashboards, the kitchen and dispatch boards, and the overall usability for the real people in section 3.
Engineering quality	Structure, typing, error handling, tests, and use of the mandated stack.
Judgement	What you prioritised, how you handled ambiguity, and how honestly you describe what's missing.
Communication	README and the state of the live app.

Good luck, and have fun with it.